Beispiele für hochperformante Jobs für ONTAP KI-Implementierungen

NetApp Solutions

NetApp
March 19, 2024

This PDF was generated from https://docs.netapp.com/de-de/netapp-solutions/ai/aicp_example_high-performance_jobs_for_ontap_ai_deployments_overview.html on March 19, 2024. Always check docs.netapp.com for the latest.

Inhalt

Beispiele für hochperformante Jobs für ONTAP KI-Implementierungen . . . 1
Beispiele für hochperformante Jobs für ONTAP KI-Implementierungen . . . 1
Single-Node-KI-Workload ausführen . . . 1
Ausführung eines synchronen, verteilten KI-Workloads . . . 5

Beispiele für hochperformante Jobs für ONTAP KI-Implementierungen

Dieser Abschnitt enthält Beispiele für verschiedene hochperformante Jobs, die ausgeführt werden können, wenn Kubernetes auf einem ONTAP AI POD implementiert wird.

Beispiele für hochperformante Jobs für ONTAP KI-Implementierungen

Dieser Abschnitt enthält Beispiele für verschiedene hochperformante Jobs, die ausgeführt werden können, wenn Kubernetes auf einem ONTAP AI POD implementiert wird.

Single-Node-KI-Workload ausführen

Um einen Single-Node-KI- und -ML-Job in Ihrem Kubernetes-Cluster auszuführen, führen Sie die folgenden Aufgaben vom Bereitstellungs-Jump-Host aus: Mit Trident lässt sich ein Daten-Volume schnell und einfach erstellen, das möglicherweise mehrere Petabyte an Daten enthält und damit für Kubernetes-Workloads zugänglich ist. Wenn ein solches Daten-Volume über einen Kubernetes Pod zugänglich sein soll, geben Sie in der Pod-Definition einfach ein PVC an. Dieser Schritt ist ein nativer Kubernetes-Betrieb. Es ist kein NetApp Fachwissen erforderlich.

> In diesem Abschnitt wird vorausgesetzt, dass Sie bereits den spezifischen KI- und ML-Workload in einem Container (im Docker Container-Format) bereitstellen, den Sie in Ihrem Kubernetes-Cluster ausführen möchten.

1. Die folgenden Beispielbefehle zeigen die Erstellung eines Kubernetes-Jobs für einen TensorFlow Benchmark-Workload, bei dem der ImageNet-Datensatz verwendet wird. Weitere Informationen zum ImageNet-Datensatz finden Sie im "ImageNet-Website".

   Dieses Beispieljob fordert acht GPUs an und kann daher auf einem einzelnen GPU-Worker-Node mit acht oder mehr GPUs ausgeführt werden. Dieser Beispieljob kann in einem Cluster eingereicht werden, für den ein Worker-Node mit acht oder mehr GPUs nicht vorhanden ist oder derzeit mit einem anderen Workload belegt ist. Wenn dies der Fall ist, bleibt der Job in einem ausstehenden Status, bis ein solcher Worker-Node verfügbar ist.

   Zusätzlich wird das Volume, das die erforderlichen Trainingsdaten enthält, zur Maximierung der Storage-Bandbreite zweimal innerhalb des POD angehängt, das diesen Job erstellt. Ein weiteres Volume wird ebenfalls im POD gemountet. Dieses zweite Volume wird zur Speicherung der Ergebnisse und Kennzahlen verwendet. Diese Volumes werden in der Jobdefinition unter Verwendung der Namen der VES referenziert. Weitere Informationen zu Kubernetes-Jobs finden Sie im "Offizielle Kubernetes-Dokumentation".

   An `emptyDir` Volumen mit einem `medium` Der Wert von `Memory` Ist in angehängt `/dev/shm` In dem POD, den dieser Beispieljob erzeugt. Die Standardgröße des `/dev/shm` Das virtuelle Volume, das automatisch durch die Docker Container-Laufzeit erstellt wird, kann manchmal nicht ausreichen, um die

Anforderungen von TensorFlow zu erfüllen. Montage an `emptyDir` Das Volumen wie im folgenden Beispiel bietet eine ausreichend große Menge `/dev/shm` Virtuelles Volume: Finden Sie weitere Informationen zu `emptyDir` Volumes, siehe "Offizielle Kubernetes-Dokumentation".

Der einzelne Container, der in dieser Beispieljobdefinition angegeben wird, wird als angegeben `securityContext > privileged` Der Wert von `true`. Dieser Wert bedeutet, dass der Container effektiv Root-Zugriff auf dem Host hat. Diese Annotation wird in diesem Fall verwendet, da der spezifische Workload ausgeführt wird und den Root-Zugriff erfordert. Insbesondere für einen Vorgang mit klarem Cache, der von dem Workload ausgeführt wird, ist ein Root-Zugriff erforderlich. Ob oder nicht `privileged: true` Eine Anmerkung ist erforderlich, abhängig von den Anforderungen des spezifischen Workloads, die Sie ausführen.

```
$ cat << EOF > ./netapp-tensorflow-single-imagenet.yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: netapp-tensorflow-single-imagenet
spec:
  backoffLimit: 5
  template:
    spec:
      volumes:
      - name: dshm
        emptyDir:
          medium: Memory
      - name: testdata-iface1
        persistentVolumeClaim:
          claimName: pb-fg-all-iface1
      - name: testdata-iface2
        persistentVolumeClaim:
          claimName: pb-fg-all-iface2
      - name: results
        persistentVolumeClaim:
          claimName: tensorflow-results
      containers:
      - name: netapp-tensorflow-py2
        image: netapp/tensorflow-py2:19.03.0
        command: ["python", "/netapp/scripts/run.py", "--
dataset_dir=/mnt/mount_0/dataset/imagenet", "--dgx_version=dgx1", "--
num_devices=8"]
        resources:
          limits:
            nvidia.com/gpu: 8
        volumeMounts:
        - mountPath: /dev/shm
          name: dshm
        - mountPath: /mnt/mount_0
```

```
          name: testdata-iface1
        - mountPath: /mnt/mount_1
          name: testdata-iface2
        - mountPath: /tmp
          name: results
        securityContext:
          privileged: true
      restartPolicy: Never
EOF
$ kubectl create -f ./netapp-tensorflow-single-imagenet.yaml
job.batch/netapp-tensorflow-single-imagenet created
$ kubectl get jobs
NAME                                       COMPLETIONS   DURATION   AGE
netapp-tensorflow-single-imagenet          0/1           24s        24s
```

2. Vergewissern Sie sich, dass der in Schritt 1 erstellte Job korrekt ausgeführt wird. Der folgende Beispielbefehl bestätigt, dass für den Job ein einzelner Pod erstellt wurde, wie in der Jobdefinition angegeben, und dass dieser Pod derzeit auf einem der GPU-Worker-Nodes ausgeführt wird.

```
$ kubectl get pods -o wide
NAME                                               READY   STATUS
RESTARTS   AGE
IP              NODE            NOMINATED NODE
netapp-tensorflow-single-imagenet-m7x92            1/1     Running     0
3m    10.233.68.61    10.61.218.154   <none>
```

3. Bestätigen Sie, dass der in Schritt 1 erstellte Job erfolgreich abgeschlossen wurde. Mit den folgenden Beispielbefehlen wird bestätigt, dass der Job erfolgreich abgeschlossen wurde.

```
$ kubectl get jobs
NAME                                          COMPLETIONS   DURATION
AGE
netapp-tensorflow-single-imagenet             1/1           5m42s
10m
$ kubectl get pods
NAME                                                  READY   STATUS
RESTARTS   AGE
netapp-tensorflow-single-imagenet-m7x92               0/1     Completed
0          11m
$ kubectl logs netapp-tensorflow-single-imagenet-m7x92
[netapp-tensorflow-single-imagenet-m7x92:00008] PMIX ERROR: NO-
PERMISSIONS in file gds_dstore.c at line 702
[netapp-tensorflow-single-imagenet-m7x92:00008] PMIX ERROR: NO-
PERMISSIONS in file gds_dstore.c at line 711
Total images/sec = 6530.59125
================ Clean Cache !!! ==================
mpirun -allow-run-as-root -np 1 -H localhost:1 bash -c 'sync; echo 1 >
/proc/sys/vm/drop_caches'
==========================================
mpirun -allow-run-as-root -np 8 -H localhost:8 -bind-to none -map-by
slot -x NCCL_DEBUG=INFO -x LD_LIBRARY_PATH -x PATH python
/netapp/tensorflow/benchmarks_190205/scripts/tf_cnn_benchmarks/tf_cnn_be
nchmarks.py --model=resnet50 --batch_size=256 --device=gpu
--force_gpu_compatible=True --num_intra_threads=1 --num_inter_threads=48
--variable_update=horovod --batch_group_size=20 --num_batches=500
--nodistortions --num_gpus=1 --data_format=NCHW --use_fp16=True
--use_tf_layers=False --data_name=imagenet --use_datasets=True
--data_dir=/mnt/mount_0/dataset/imagenet
--datasets_parallel_interleave_cycle_length=10
--datasets_sloppy_parallel_interleave=False --num_mounts=2
--mount_prefix=/mnt/mount_%d --datasets_prefetch_buffer_size=2000
--datasets_use_prefetch=True --datasets_num_private_threads=4
--horovod_device=gpu >
/tmp/20190814_105450_tensorflow_horovod_rdma_resnet50_gpu_8_256_b500_ima
genet_nodistort_fp16_r10_m2_nockpt.txt 2>&1
```

4. **Optional:** Aufräumen von Auftragsartefakten. Die folgenden Beispielbefehle zeigen das Löschen des in Schritt 1 erstellten Jobobjekts.

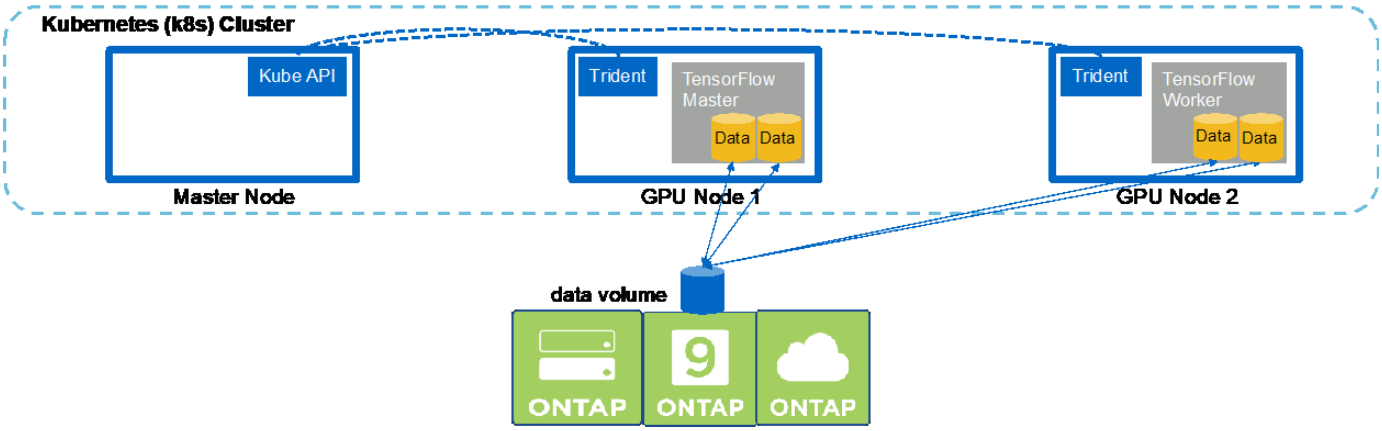
   Wenn Sie das Jobobjekt löschen, löscht Kubernetes automatisch alle zugehörigen Pods.

```
$ kubectl get jobs
NAME                                                COMPLETIONS   DURATION
AGE
netapp-tensorflow-single-imagenet                   1/1           5m42s
10m
$ kubectl get pods
NAME                                                      READY   STATUS
RESTARTS   AGE
netapp-tensorflow-single-imagenet-m7x92                   0/1     Completed
0          11m
$ kubectl delete job netapp-tensorflow-single-imagenet
job.batch "netapp-tensorflow-single-imagenet" deleted
$ kubectl get jobs
No resources found.
$ kubectl get pods
No resources found.
```

Ausführung eines synchronen, verteilten KI-Workloads

Um einen synchronen AI- und ML-Job mit mehreren Nodes in Ihrem Kubernetes-Cluster auszuführen, führen Sie die folgenden Aufgaben auf dem Jump-Host der Implementierung durch. Mit diesem Prozess können Sie auf einem NetApp Volume gespeicherte Daten nutzen und mehr GPUs verwenden, als ein einzelner Worker-Node bieten kann. Die folgende Abbildung zeigt einen synchronen, verteilten KI-Job.

> Synchrone, verteilte Jobs können dazu beitragen, die Performance und die Trainingsgenauigkeit im Vergleich zu asynchronen, verteilten Jobs zu steigern. Eine Diskussion über die vor- und Nachteile von synchronen Jobs gegenüber asynchronen Jobs geht nicht in dieses Dokument über.



1. Die folgenden Beispielbefehle zeigen die Erstellung eines Workers, der an der synchronen, verteilten Ausführung desselben TensorFlow Benchmark-Jobs beteiligt ist, der auf einem einzelnen Node im Beispiel im Abschnitt ausgeführt wurde "Single-Node-KI-Workload ausführen". In diesem speziellen Beispiel wird nur ein einziger Worker bereitgestellt, da der Job über zwei Worker-Nodes ausgeführt wird.

Dieses Beispiel für die Implementierung eines Mitarbeiters fordert acht GPUs an und kann so auf einem einzelnen GPU-Worker-Node mit acht oder mehr GPUs ausgeführt werden. Wenn Ihre GPU-Worker-Nodes mehr als acht GPUs aufweisen, um die Performance zu maximieren, könnte die Anzahl dieser GPUs erhöht werden, die der Funktion der Worker-Nodes entsprechen. Weitere Informationen über Kubernetes-Implementierungen finden Sie im "Offizielle Kubernetes-Dokumentation".

In diesem Beispiel wird eine Kubernetes-Implementierung erstellt, da dieser spezifische Container-Worker niemals eigenständig abgeschlossen sein würde. Daher macht es keinen Sinn, es durch die Verwendung des Kubernetes Job-Konstrukts zu implementieren. Wenn Ihr Mitarbeiter für sich selbst konzipiert oder geschrieben wurde, ist es möglicherweise sinnvoll, das Job-Konstrukt für die Bereitstellung Ihres Mitarbeiters zu verwenden.

Dem POD, der in diesem Beispiel der Implementierungsspezifikation angegeben ist, wird eine zugewiesen `hostNetwork` Der Wert von `true`. Dieser Wert bedeutet, dass der Pod den Netzwerk-Stack des Host-Mitarbeiters-Nodes anstelle des virtuellen Netzwerk-Stacks verwendet, den Kubernetes normalerweise für jeden Pod erstellt. Diese Annotation wird in diesem Fall verwendet, da der spezifische Workload auf Open MPI, NCCL und Horovod angewiesen ist, um den Workload in einer synchronen, verteilten Art und Weise auszuführen. Daher ist für diese Lösung der Zugriff auf den Host-Netzwerk-Stack erforderlich. Eine Diskussion über Open MPI, NCCL und Horovod geht nicht in dieses Dokument über. Ob oder nicht `hostNetwork: true` Eine Anmerkung ist erforderlich, abhängig von den Anforderungen des spezifischen Workloads, die Sie ausführen. Weitere Informationen zum `hostNetwork` Feld, siehe "Offizielle Kubernetes-Dokumentation".

```
$ cat << EOF > ./netapp-tensorflow-multi-imagenet-worker.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: netapp-tensorflow-multi-imagenet-worker
spec:
  replicas: 1
  selector:
    matchLabels:
      app: netapp-tensorflow-multi-imagenet-worker
  template:
    metadata:
      labels:
        app: netapp-tensorflow-multi-imagenet-worker
    spec:
      hostNetwork: true
      volumes:
      - name: dshm
        emptyDir:
          medium: Memory
      - name: testdata-iface1
        persistentVolumeClaim:
          claimName: pb-fg-all-iface1
      - name: testdata-iface2
        persistentVolumeClaim:
          claimName: pb-fg-all-iface2
```

```
      - name: results
        persistentVolumeClaim:
          claimName: tensorflow-results
      containers:
      - name: netapp-tensorflow-py2
        image: netapp/tensorflow-py2:19.03.0
        command: ["bash", "/netapp/scripts/start-slave-multi.sh",
"22122"]
        resources:
          limits:
            nvidia.com/gpu: 8
        volumeMounts:
        - mountPath: /dev/shm
          name: dshm
        - mountPath: /mnt/mount_0
          name: testdata-iface1
        - mountPath: /mnt/mount_1
          name: testdata-iface2
        - mountPath: /tmp
          name: results
        securityContext:
          privileged: true
EOF
$ kubectl create -f ./netapp-tensorflow-multi-imagenet-worker.yaml
deployment.apps/netapp-tensorflow-multi-imagenet-worker created
$ kubectl get deployments
NAME                                      DESIRED   CURRENT   UP-TO-DATE
AVAILABLE   AGE
netapp-tensorflow-multi-imagenet-worker   1         1         1
1           4s
```

2. Bestätigen Sie, dass die in Schritt 1 erstellte Workers-Bereitstellung erfolgreich gestartet wurde. Die folgenden Beispielbefehle bestätigen, dass ein Pod für einzelne Mitarbeiter für die Implementierung erstellt wurde, wie in der Implementierungsdefinition angegeben, und dass dieser Pod derzeit auf einem der GPU-Worker-Nodes ausgeführt wird.

```
$ kubectl get pods -o wide
NAME                                                      READY
STATUS    RESTARTS   AGE
IP              NODE            NOMINATED NODE
netapp-tensorflow-multi-imagenet-worker-654fc7f486-v6725   1/1
Running   0          60s   10.61.218.154   10.61.218.154   <none>
$ kubectl logs netapp-tensorflow-multi-imagenet-worker-654fc7f486-v6725
22122
```

3. Kubernetes-Job für einen Master erstellen, der zu Beginn startet, an dem er teilnimmt und die Ausführung des synchronen Jobs mit mehreren Nodes verfolgt. Die folgenden Beispielbefehle erzeugen einen Master, der abstartet, an dem teilnimmt und die synchrone, verteilte Ausführung desselben TensorFlow-Benchmark-Jobs verfolgt, der auf einem einzelnen Node im Beispiel im Abschnitt ausgeführt wurde "Single-Node-KI-Workload ausführen".

   Dieses Beispiel-Master-Job fordert acht GPUs an, wodurch auf einem einzelnen GPU-Worker-Node mit acht oder mehr GPUs ausgeführt werden kann. Wenn Ihre GPU-Worker-Nodes mehr als acht GPUs aufweisen, um die Performance zu maximieren, könnte die Anzahl dieser GPUs erhöht werden, die der Funktion der Worker-Nodes entsprechen.

   Der Master-Pod, der in dieser Beispiel-Jobdefinition angegeben wird, wird A zugewiesen `hostNetwork` Der Wert von `true`, So wie der Arbeiter POD wurde ein `hostNetwork` Der Wert von `true` In Schritt 1. Weitere Informationen dazu, warum dieser Wert notwendig ist, finden Sie in Schritt 1.

```
$ cat << EOF > ./netapp-tensorflow-multi-imagenet-master.yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: netapp-tensorflow-multi-imagenet-master
spec:
  backoffLimit: 5
  template:
    spec:
      hostNetwork: true
      volumes:
      - name: dshm
        emptyDir:
          medium: Memory
      - name: testdata-iface1
        persistentVolumeClaim:
          claimName: pb-fg-all-iface1
      - name: testdata-iface2
        persistentVolumeClaim:
          claimName: pb-fg-all-iface2
      - name: results
        persistentVolumeClaim:
          claimName: tensorflow-results
      containers:
      - name: netapp-tensorflow-py2
        image: netapp/tensorflow-py2:19.03.0
        command: ["python", "/netapp/scripts/run.py", "--
dataset_dir=/mnt/mount_0/dataset/imagenet", "--port=22122", "--
num_devices=16", "--dgx_version=dgx1", "--
nodes=10.61.218.152,10.61.218.154"]
        resources:
          limits:
            nvidia.com/gpu: 8
```

```
        volumeMounts:
        - mountPath: /dev/shm
          name: dshm
        - mountPath: /mnt/mount_0
          name: testdata-iface1
        - mountPath: /mnt/mount_1
          name: testdata-iface2
        - mountPath: /tmp
          name: results
        securityContext:
          privileged: true
      restartPolicy: Never
EOF
$ kubectl create -f ./netapp-tensorflow-multi-imagenet-master.yaml
job.batch/netapp-tensorflow-multi-imagenet-master created
$ kubectl get jobs
NAME                                        COMPLETIONS   DURATION   AGE
netapp-tensorflow-multi-imagenet-master   0/1           25s        25s
```

4. Vergewissern Sie sich, dass der in Schritt 3 erstellte Master-Job korrekt ausgeführt wird. Der folgende Beispielbefehl bestätigt, dass für den Job ein einzelner Master-Pod erstellt wurde, wie in der Jobdefinition angegeben, und dass dieser Pod derzeit auf einem der GPU-Worker-Nodes ausgeführt wird. Sie sollten auch sehen, dass der Worker Pod, den Sie ursprünglich in Schritt 1 gesehen haben, noch läuft und dass die Master- und Worker-Pods auf unterschiedlichen Nodes ausgeführt werden.

```
$ kubectl get pods -o wide
NAME                                                          READY
STATUS    RESTARTS   AGE
IP              NODE            NOMINATED NODE
netapp-tensorflow-multi-imagenet-master-ppwwj               1/1
Running   0          45s   10.61.218.152   10.61.218.152   <none>
netapp-tensorflow-multi-imagenet-worker-654fc7f486-v6725   1/1
Running   0          26m   10.61.218.154   10.61.218.154   <none>
```

5. Vergewissern Sie sich, dass der in Schritt 3 erstellte Masterjob erfolgreich abgeschlossen wurde. Mit den folgenden Beispielbefehlen wird bestätigt, dass der Job erfolgreich abgeschlossen wurde.

```
$ kubectl get jobs
NAME                                        COMPLETIONS   DURATION   AGE
netapp-tensorflow-multi-imagenet-master   1/1           5m50s      9m18s
$ kubectl get pods
NAME                                                          READY
STATUS      RESTARTS   AGE
netapp-tensorflow-multi-imagenet-master-ppwwj               0/1
Completed   0          9m38s
```

```
netapp-tensorflow-multi-imagenet-worker-654fc7f486-v6725   1/1
Running     0          35m
$ kubectl logs netapp-tensorflow-multi-imagenet-master-ppwwj
[10.61.218.152:00008] WARNING: local probe returned unhandled
shell:unknown assuming bash
rm: cannot remove '/lib': Is a directory
[10.61.218.154:00033] PMIX ERROR: NO-PERMISSIONS in file gds_dstore.c at
line 702
[10.61.218.154:00033] PMIX ERROR: NO-PERMISSIONS in file gds_dstore.c at
line 711
[10.61.218.152:00008] PMIX ERROR: NO-PERMISSIONS in file gds_dstore.c at
line 702
[10.61.218.152:00008] PMIX ERROR: NO-PERMISSIONS in file gds_dstore.c at
line 711
Total images/sec = 12881.33875
================ Clean Cache !!! ==================
mpirun -allow-run-as-root -np 2 -H 10.61.218.152:1,10.61.218.154:1 -mca
pml ob1 -mca btl ^openib -mca btl_tcp_if_include enp1s0f0 -mca
plm_rsh_agent ssh -mca plm_rsh_args "-p 22122" bash -c 'sync; echo 1 >
/proc/sys/vm/drop_caches'
==========================================
mpirun -allow-run-as-root -np 16 -H 10.61.218.152:8,10.61.218.154:8
-bind-to none -map-by slot -x NCCL_DEBUG=INFO -x LD_LIBRARY_PATH -x PATH
-mca pml ob1 -mca btl ^openib -mca btl_tcp_if_include enp1s0f0 -x
NCCL_IB_HCA=mlx5 -x NCCL_NET_GDR_READ=1 -x NCCL_IB_SL=3 -x
NCCL_IB_GID_INDEX=3 -x
NCCL_SOCKET_IFNAME=enp5s0.3091,enp12s0.3092,enp132s0.3093,enp139s0.3094
-x NCCL_IB_CUDA_SUPPORT=1 -mca orte_base_help_aggregate 0 -mca
plm_rsh_agent ssh -mca plm_rsh_args "-p 22122" python
/netapp/tensorflow/benchmarks_190205/scripts/tf_cnn_benchmarks/tf_cnn_be
nchmarks.py --model=resnet50 --batch_size=256 --device=gpu
--force_gpu_compatible=True --num_intra_threads=1 --num_inter_threads=48
--variable_update=horovod --batch_group_size=20 --num_batches=500
--nodistortions --num_gpus=1 --data_format=NCHW --use_fp16=True
--use_tf_layers=False --data_name=imagenet --use_datasets=True
--data_dir=/mnt/mount_0/dataset/imagenet
--datasets_parallel_interleave_cycle_length=10
--datasets_sloppy_parallel_interleave=False --num_mounts=2
--mount_prefix=/mnt/mount_%d --datasets_prefetch_buffer_size=2000 --
datasets_use_prefetch=True --datasets_num_private_threads=4
--horovod_device=gpu >
/tmp/20190814_161609_tensorflow_horovod_rdma_resnet50_gpu_16_256_b500_im
agenet_nodistort_fp16_r10_m2_nockpt.txt 2>&1
```

6. Löschen Sie die Mitarbeiterbereitstellung, wenn Sie sie nicht mehr benötigen. Die folgenden Beispielbefehle zeigen das Löschen des in Schritt 1 erstellten Workers Deployment-Objekts.

Wenn Sie das Bereitstellungsobjekt für Mitarbeiter löschen, löscht Kubernetes automatisch alle zugehörigen „Worker“-Pods.

```
$ kubectl get deployments
NAME                                        DESIRED   CURRENT   UP-TO-DATE
AVAILABLE   AGE
netapp-tensorflow-multi-imagenet-worker   1         1         1
1           43m
$ kubectl get pods
NAME                                                        READY
STATUS      RESTARTS   AGE
netapp-tensorflow-multi-imagenet-master-ppwwj              0/1
Completed   0          17m
netapp-tensorflow-multi-imagenet-worker-654fc7f486-v6725   1/1
Running     0          43m
$ kubectl delete deployment netapp-tensorflow-multi-imagenet-worker
deployment.extensions "netapp-tensorflow-multi-imagenet-worker" deleted
$ kubectl get deployments
No resources found.
$ kubectl get pods
NAME                                            READY   STATUS
RESTARTS   AGE
netapp-tensorflow-multi-imagenet-master-ppwwj   0/1     Completed   0
18m
```

7. **Optional:** Säubern Sie die Master Job Artefakte. Die folgenden Beispielbefehle zeigen das Löschen des in Schritt 3 erstellten Master-Jobobjekts.

   Wenn Sie das Master-Job-Objekt löschen, löscht Kubernetes automatisch alle zugehörigen Master-Pods.

```
$ kubectl get jobs
NAME                                        COMPLETIONS   DURATION   AGE
netapp-tensorflow-multi-imagenet-master   1/1           5m50s      19m
$ kubectl get pods
NAME                                            READY   STATUS
RESTARTS   AGE
netapp-tensorflow-multi-imagenet-master-ppwwj   0/1     Completed   0
19m
$ kubectl delete job netapp-tensorflow-multi-imagenet-master
job.batch "netapp-tensorflow-multi-imagenet-master" deleted
$ kubectl get jobs
No resources found.
$ kubectl get pods
No resources found.
```

Copyright-Informationen

Copyright © 2024 NetApp. Alle Rechte vorbehalten. Gedruckt in den USA. Dieses urheberrechtlich geschützte Dokument darf ohne die vorherige schriftliche Genehmigung des Urheberrechtsinhabers in keiner Form und durch keine Mittel – weder grafische noch elektronische oder mechanische, einschließlich Fotokopieren, Aufnehmen oder Speichern in einem elektronischen Abrufsystem – auch nicht in Teilen, vervielfältigt werden.

Software, die von urheberrechtlich geschütztem NetApp Material abgeleitet wird, unterliegt der folgenden Lizenz und dem folgenden Haftungsausschluss:

DIE VORLIEGENDE SOFTWARE WIRD IN DER VORLIEGENDEN FORM VON NETAPP ZUR VERFÜGUNG GESTELLT, D. H. OHNE JEGLICHE EXPLIZITE ODER IMPLIZITE GEWÄHRLEISTUNG, EINSCHLIESSLICH, JEDOCH NICHT BESCHRÄNKT AUF DIE STILLSCHWEIGENDE GEWÄHRLEISTUNG DER MARKTGÄNGIGKEIT UND EIGNUNG FÜR EINEN BESTIMMTEN ZWECK, DIE HIERMIT AUSGESCHLOSSEN WERDEN. NETAPP ÜBERNIMMT KEINERLEI HAFTUNG FÜR DIREKTE, INDIREKTE, ZUFÄLLIGE, BESONDERE, BEISPIELHAFTE SCHÄDEN ODER FOLGESCHÄDEN (EINSCHLIESSLICH, JEDOCH NICHT BESCHRÄNKT AUF DIE BESCHAFFUNG VON ERSATZWAREN ODER -DIENSTLEISTUNGEN, NUTZUNGS-, DATEN- ODER GEWINNVERLUSTE ODER UNTERBRECHUNG DES GESCHÄFTSBETRIEBS), UNABHÄNGIG DAVON, WIE SIE VERURSACHT WURDEN UND AUF WELCHER HAFTUNGSTHEORIE SIE BERUHEN, OB AUS VERTRAGLICH FESTGELEGTER HAFTUNG, VERSCHULDENSUNABHÄNGIGER HAFTUNG ODER DELIKTSHAFTUNG (EINSCHLIESSLICH FAHRLÄSSIGKEIT ODER AUF ANDEREM WEGE), DIE IN IRGENDEINER WEISE AUS DER NUTZUNG DIESER SOFTWARE RESULTIEREN, SELBST WENN AUF DIE MÖGLICHKEIT DERARTIGER SCHÄDEN HINGEWIESEN WURDE.

NetApp behält sich das Recht vor, die hierin beschriebenen Produkte jederzeit und ohne Vorankündigung zu ändern. NetApp übernimmt keine Verantwortung oder Haftung, die sich aus der Verwendung der hier beschriebenen Produkte ergibt, es sei denn, NetApp hat dem ausdrücklich in schriftlicher Form zugestimmt. Die Verwendung oder der Erwerb dieses Produkts stellt keine Lizenzierung im Rahmen eines Patentrechts, Markenrechts oder eines anderen Rechts an geistigem Eigentum von NetApp dar.

Das in diesem Dokument beschriebene Produkt kann durch ein oder mehrere US-amerikanische Patente, ausländische Patente oder anhängige Patentanmeldungen geschützt sein.

ERLÄUTERUNG ZU „RESTRICTED RIGHTS“: Nutzung, Vervielfältigung oder Offenlegung durch die US-Regierung unterliegt den Einschränkungen gemäß Unterabschnitt (b)(3) der Klausel „Rights in Technical Data – Noncommercial Items“ in DFARS 252.227-7013 (Februar 2014) und FAR 52.227-19 (Dezember 2007).

Die hierin enthaltenen Daten beziehen sich auf ein kommerzielles Produkt und/oder einen kommerziellen Service (wie in FAR 2.101 definiert) und sind Eigentum von NetApp, Inc. Alle technischen Daten und die Computersoftware von NetApp, die unter diesem Vertrag bereitgestellt werden, sind gewerblicher Natur und wurden ausschließlich unter Verwendung privater Mittel entwickelt. Die US-Regierung besitzt eine nicht ausschließliche, nicht übertragbare, nicht unterlizenzierbare, weltweite, limitierte unwiderrufliche Lizenz zur Nutzung der Daten nur in Verbindung mit und zur Unterstützung des Vertrags der US-Regierung, unter dem die Daten bereitgestellt wurden. Sofern in den vorliegenden Bedingungen nicht anders angegeben, dürfen die Daten ohne vorherige schriftliche Genehmigung von NetApp, Inc. nicht verwendet, offengelegt, vervielfältigt, geändert, aufgeführt oder angezeigt werden. Die Lizenzrechte der US-Regierung für das US-Verteidigungsministerium sind auf die in DFARS-Klausel 252.227-7015(b) (Februar 2014) genannten Rechte beschränkt.

Markeninformationen

NETAPP, das NETAPP Logo und die unter http://www.netapp.com/TM aufgeführten Marken sind Marken von NetApp, Inc. Andere Firmen und Produktnamen können Marken der jeweiligen Eigentümer sein.